

APPLIED KNOWLEDGE LAB

F0.2 - Foundation e ingesta inicial de SAT

Documento tecnico y pedagogico de referencia

Proyecto: aklab

Primer caso real: Gredos ERP

Fuente inicial: SAT.mdb

Estado: F0.2 cerrada

Resumen ejecutivo

F0.2 demuestra que AKLab puede ingerir una fuente legacy real, conservarla con fidelidad, normalizar conocimiento y mantener evidencia e idempotencia sin utilizar IA generativa. La arquitectura queda preparada para empezar F1 desde un KnowledgeSpace vacío y, posteriormente, comparar técnicas de IA contra este baseline determinista.

Indicador	Resultado
Fuente real	SAT.mdb / Microsoft Access
CONSULTAS	3.249 registros
HISTORICO	10.369 registros
Total exportado	13.618 registros
Tests de cierre	6 passed
LLM / RAG	No usados en F0.2
Siguiente fase	F1 - CRUD y entrada manual desde cero

Mapa de la fase

```
SAT.mdb
-> mdbtools / JSONL neutral
-> GredosSatAdapter
-> SourceItem
-> GredosSatNormalizer
-> KnowledgeItem + Evidence
```

Indice de referencia

- 1. Objetivo y principios
- 2. Arquitectura y modelo canonico
- 3. Extraccion de SAT y fidelidad de datos
- 4. CONSULTAS: normalizacion e idempotencia
- 5. HISTORICO: relacion, huérfanos y decision
- 6. Confianza, evidencia y verdad
- 7. Pruebas y resultados
- 8. Decisiones ADR
- 9. Riesgos pospuestos y limite de F0.2
- 10. Punto exacto para continuar con F1

1. Objetivo de F0.2

F0.2 debía demostrar, antes de introducir LLM, embeddings o RAG, que Applied Knowledge Lab podía recibir una fuente legacy real, conservarla sin pérdida, transformarla a un modelo canónico propio y mantener trazabilidad entre conocimiento y evidencia.

El objetivo no era consultar Access de forma permanente. El objetivo era demostrar este flujo:

```
fuelle externa
-> extractor/adaptador
-> SourceItem
-> normalizacion determinista
-> KnowledgeItem
-> Evidence
```

Con cuatro propiedades obligatorias:

- reproducibilidad;
- trazabilidad;
- idempotencia;
- independencia del producto/fuente.

F0.2 se ha desarrollado deliberadamente **sin LLM**. Esto proporciona un baseline limpio para comparar las técnicas de IA de fases posteriores.

2. Principio arquitectónico central

SAT.mdb no es el modelo interno de AKLab.

Gredos es el primer caso de uso, pero el núcleo utiliza conceptos genéricos:

- `KnowledgeSpace`
- `Source`
- `ImportRun`
- `SourceItem`
- `KnowledgeItem`
- `KnowledgeSection`
- `KnowledgeAttribute`
- `Evidence`

La lógica específica de SAT vive detrás del adaptador Gredos SAT.

La pregunta permanente sigue siendo:

¿Esto pertenece al motor genérico o al adaptador/producto Gredos?

3. Estructura actual del repositorio

```
aklab/  
  src/applied_knowledge/  
    domain/  
    storage/  
    ingestion/  
    adapters/  
      gredos/  
      sat/  
  tests/  
  scripts/  
  docs/  
    adr/  
  data/  
    raw/  
    generated/
```

El paquete Python sigue llamándose `applied_knowledge`. El repositorio/directorio se llama `aklab`.

4. Separacion entre dominio y persistencia

Los modelos de dominio y los modelos SQLAlchemy son capas distintas.

Esto evita que SQLite o SQLAlchemy se conviertan accidentalmente en el modelo conceptual del producto.

SQLite es la implementación inicial de persistencia, no un requisito arquitectónico permanente.

5. SourceAdapter y frontera con la fuente

El núcleo consume `SourceRecord`.

El adaptador tiene una responsabilidad limitada:

```
fuelle externa  
-> SourceRecord
```

No crea `KnowledgeItem`.

En el caso de SAT:

```
SAT.mdb  
-> SatExtractor  
-> GredosSatAdapter  
-> SourceRecord
```

La lectura concreta de Access queda aislada detrás de `SatExtractor`.

Implementaciones actuales:

- ``MdbToolsExtractor``
- ``NeutralJsonlExtractor``

Esto permite que macOS/Linux utilicen `mbdtools` y que otras plataformas puedan utilizar otro extractor sin modificar el núcleo.

6. Exportacion neutral

Se decidió introducir una frontera neutral reproducible:

```
SAT.mdb
-> mdbtools
-> JSONL
-> AKLab
```

El formato neutral validado contiene:

Tabla	Registros
CONSULTAS	3.249
HISTORICO	10.369
Total	13.618

El `manifest.json` conserva también la huella SHA-256 del fichero fuente.

SHA-256 validado de la copia de SAT utilizada:

```
e682ec32475183ec7eb0569d048edc4aa0deff6189ee946c2558a315fe33cc85
```

7. Fidelidad de texto y encoding

La salida real de `mdb-export` se verificó como UTF-8 estricto.

Resultado:

```
UTF-8: OK
replacement chars: 0
```

Se comprobaron además textos con:

- acentos;
- saltos de línea reales;
- texto multilínea;
- rutas DOS/Windows;
- espacios no separables.

El extractor usa `errors="strict"`. Si en el futuro aparece texto no decodificable, la importación debe fallar de forma visible en vez de reemplazar silenciosamente caracteres.

8. Preservacion de NULL

Durante la validación se descubrió que una exportación convencional podía confundir valores NULL de Access con cadenas vacías.

Se corrigió utilizando un sentinel explícito durante `mdb-export` y convirtiéndolo posteriormente a `None`.

En CONSULTAS se observaron:

```
NULL: 12095
cadenas vacias: 0
```

Esta corrección es importante porque `raw_content` debe representar la fuente con fidelidad.

9. CONSULTAS - esquema real

CONSULTAS contiene 3.249 registros y 22 campos:

```
ID_CONSULTA
CABECERA
TECNICO
FECHA
PROGRAMA
MODULO
SINTOMAS
ENTORNO
SOLUCION
OBSERVACIONES
BORRADO
REPARA
ULTIMA
ID_PRI
ID_SIT
ESPERA
QUIEN
IMPRESO
PUBLICADO
FECHAAVIS0
HORAAVIS0
CodigoWeb
```

Los campos `FECHAAVIS0`, `HORAAVIS0` y `Codigoweb` no estaban en la reconstrucción conceptual inicial. El modelo no necesitó cambiar para conservarlos porque `SourceItem.raw_content` mantiene la fila completa.

Esta es una validación práctica de una decisión fundamental: **la normalización no debe destruir la evidencia original**.

10. Normalizacion inicial de CONSULTAS

F0.2 utiliza una normalización conservadora y determinista.

Una fila de CONSULTAS produce inicialmente:

```
KnowledgeItem
  kind = support_case
  title = CABECERA
```

Secciones:

```
symptom      <- SINTOMAS
environment  <- ENTORNO
solution     <- SOLUCION
notes        <- OBSERVACIONES
```

Atributos:

```
program <- PROGRAMA
module  <- MODULO
```

Y una evidencia:

```
Evidence
evidence_type = source_record
locator = CONSULTAS:<ID_CONSULTA>
```

No se extraen todavía errores, parámetros, versiones o procedimientos como objetos separados. Eso llegará en fases posteriores.

11. Ejemplo real: CONSULTAS:1

El primer registro real inspeccionado fue:

```
Error 75 al entrar en gestion con Windows 95.
```

En su contenido aparecen elementos como:

- Error 75;
- Windows 95;
- versión 8.855B;
- parámetro `WIN95=1`;
- fichero `GENERAL.PRM`;
- un procedimiento de configuración.

F0.2 no intenta convertir todavía esos elementos en entidades independientes.

El valor pedagógico del ejemplo es mostrar que una única fila fuente puede contener muchas futuras unidades de conocimiento:

```
SourceItem CONSULTAS:1
-> error
-> parametro
-> fichero de configuracion
-> version
-> procedimiento
```

Esto confirma la decisión original de no identificar automáticamente 1 SourceItem = 1 KnowledgeItem como regla universal.

12. Idempotencia

Se importó una muestra controlada de 10 consultas reales.

Primera ejecución:

```
new=10
changed=0
unchanged=0
knowledge_items=10
```

Segunda ejecución:

```
new=0
changed=0
unchanged=10
knowledge_items=0
```

La base contenía después:

- 2 `ImportRun`;
- 10 `SourceItem`;
- 10 `KnowledgeItem`;
- 10 `Evidence`.

Esto demuestra que el historial de importación está separado del estado actual del conocimiento.

13. Identificadores y checksum

Los identificadores externos se conservan con namespace de tabla:

```
CONSULTAS:1
HISTORICO:70
```

No se presupone que los IDs sean consecutivos.

Los identificadores internos de `SourceItem` y `KnowledgeItem` se generan de forma estable.

El checksum canónico de cada registro permite distinguir:

- `new`
- `changed`
- `unchanged`

14. ImportRun

Cada importación crea un `ImportRun` independiente con:

- fuente;
- adaptador;
- versión del adaptador;
- fingerprint de la fuente;
- estado;
- estadísticas;
- timestamps.

Esto permite responder posteriormente preguntas como:

- qué fichero produjo un estado;
- qué versión del adaptador se utilizó;

- cuántos registros cambiaron;
- si una ejecución falló.

15. Verdad, confianza y evidencia

Durante F0.2 se corrigió una ambigüedad importante.

Una importación determinista puede demostrar que **SAT afirma algo**, pero no que esa afirmación sea objetivamente cierta o siga vigente.

Por tanto:

```
KnowledgeItem
  status = active
  confidence = NULL

Evidence
  confidence = 1.0
```

`Evidence.confidence = 1.0` significa que el vínculo con la fuente es cierto.

No significa que la afirmación de SAT sea una verdad absoluta.

Esto será fundamental cuando entren nuevas fuentes como:

- código fuente;
- documentación;
- conocimiento manual;
- históricos de soporte.

Las fuentes podrán apoyarse o contradecirse.

16. HISTORICO - hallazgos reales

HISTORICO contiene 10.369 registros.

Campos:

```
ID_HISTORICO
ID_BOLETIN
FECHA
HORA
DESCRIPCION
EXPLICACION
ID_TECNICO
```

Resultados del análisis:

Métrica	Resultado
HISTORICO total	10.369
ID_HISTORICO no nulos	10.369

ID_HISTORICO duplicados	0
Históricos vinculados a CONSULTAS actual	10.188
Históricos huérfanos	181
Consultas con histórico	2.641
Consultas sin histórico	608
Media por consulta con histórico	3,86
Máximo observado	149

La consulta con más histórico observada fue CONSULTAS:314, con 149 entradas.

17. Decision sobre HISTORICO

Los históricos contienen información heterogénea.

Ejemplos reales incluyen:

```
Cambio de Responsable. De JESUS a IÑAKI
version 8.855g tema facturacion impresion s/n
Acerca del BACKUP
```

Por tanto, una fila HISTORICO **no se convierte automáticamente en KnowledgeItem**.

La decisión F0.2 es:

```
HISTORICO:<ID_HISTORICO>
-> SourceItem independiente
-> parent_external_id = CONSULTAS:<ID_BOLETIN>
```

parent_external_id pertenece al modelo genérico de fuente, no a Gredos.

No se utiliza una foreign key física al padre porque la fuente real contiene 181 referencias huérfanas.

Los huérfanos se preservan. AKLab no los elimina, corrige ni conecta con un padre inventado.

18. Por que HISTORICO no es conocimiento automatico

Un histórico puede representar:

- una operación administrativa;
- una reasignación de técnico;
- un comentario temporal;
- una versión;
- una solución nueva;

- una aclaración;
- conocimiento técnico reutilizable.

La fuente debe conservarse primero.

Más adelante una fase de clasificación/extracción podrá producir candidatos:

```
HISTORICO SourceItem
-> clasificacion IA
-> KnowledgeCandidate
-> revision humana
-> KnowledgeItem
```

Esto mantiene clara la diferencia entre **dato fuente** y **conocimiento confirmado**.

19. Pruebas al cierre

La suite focalizada de F0.2 contiene 6 tests y está verde:

```
6 passed
```

Cubre:

- importación idempotente;
- actualización de un registro modificado sin duplicación;
- normalización determinista de CONSULTAS;
- semántica de `status/confidence`;
- emisión de CONSULTAS e HISTORICO por el adaptador;
- `parent\_external\_id` para históricos;
- preservación de históricos huérfanos;
- ausencia de KnowledgeItem automático para HISTORICO.

También se realizó una prueba end-to-end neutral:

Primera importación:

```
ImportResult(new=3, changed=0, unchanged=0, knowledge_items=1)
```

Segunda importación:

```
ImportResult(new=0, changed=0, unchanged=3, knowledge_items=0)
```

Resultado de almacenamiento:

```
import_runs      2
source_items     3
knowledge_items   1
evidence         1
```

Con relaciones:

```
CONSULTAS:7      -> parent = NULL
HISTORICO:70     -> parent = CONSULTAS:7
HISTORICO:71     -> parent = CONSULTAS:999
```

El último caso demuestra que un huérfano se conserva sin inventar el padre.

20. Decisiones arquitectonicas documentadas

F0.2 deja tres ADR principales:

ADR-0001 - Access es dependencia del adaptador

Access/mdbtools no pertenece al núcleo.

ADR-0002 - Confidence no significa verdad de la fuente

La confianza del vínculo de evidencia y la certeza epistemológica son conceptos diferentes.

ADR-0003 - HISTORICO es fuente, no conocimiento automático

Cada histórico se conserva individualmente y puede enlazarse a su consulta mediante `parent_external_id`.

21. Lo que F0.2 no hace deliberadamente

Todavía no existen:

- llamadas a LLM;
- embeddings;
- vector search;
- RAG;
- hybrid search;
- reranking;
- query rewriting;
- Entity/Relation;
- KnowledgeCandidate;
- KnowledgeRevision;
- GraphRAG;
- agentes;
- fine-tuning.

Tampoco se han incorporado aún como fuentes de conocimiento:

- `DOCUMENTOS`;
- `IMAGENES`;
- código fuente VB6;
- otras tablas operativas de SAT.

Esto no es una carencia: es la frontera deliberada de F0.2.

22. Riesgos y temas conscientemente pospuestos

Hay varias necesidades futuras que el diseño ya permite estudiar, pero que no se resuelven todavía:

- detección de registros eliminados/orphaned entre importaciones sucesivas;
- invalidación/re-normalización cuando cambie la versión del normalizador;
- migraciones de esquema cuando exista una base persistente que no pueda regenerarse;
- extracción 1:N real desde una fuente a varios KnowledgeItem;
- eliminación de KnowledgeItem antiguos si una futura normalización deja de producirlos;
- autoridad relativa y contradicciones entre fuentes.

Estas cuestiones deben resolverse cuando exista un caso funcional que las necesite, no mediante abstracciones prematuras.

23. Criterio de éxito de F0.2

F0.2 puede considerarse conseguida porque ya podemos afirmar que:

1. SAT puede leerse de forma reproducible.
2. Access no forma parte del núcleo.
3. La fuente original se conserva íntegramente.
4. Los NULL se preservan.
5. La importación es idempotente.
6. Existe historial de importaciones.
7. CONSULTAS puede normalizarse sin IA.
8. Cada KnowledgeItem puede rastrearse hasta su evidencia.
9. HISTORICO puede conservarse sin confundirlo con conocimiento.
10. Las referencias huérfanas de la fuente no se destruyen ni se inventan.
11. El modelo sigue siendo genérico y no depende de nombres como `ID\_BOLETIN` fuera del adaptador.

24. Punto exacto para continuar

El siguiente paso natural es **F1 - CRUD y entrada manual de conocimiento**.

La prueba conceptual de F1 debe ser crear un KnowledgeSpace completamente vacío, sin SAT, e introducir conocimiento nuevo manualmente.

Objetivo:

```
KnowledgeSpace vacio
-> entrada manual
-> KnowledgeItem
-> secciones / atributos
-> evidencia de origen manual
-> consulta/inspeccion basica
```

Esto demostrará el segundo gran requisito arquitectónico: AKLab no sólo importa conocimiento existente; también puede comenzar desde cero.

Después de F1 podremos construir el baseline de búsqueda clásica de F2.

25. Resumen pedagogico

Lo más importante aprendido en F0.2 no es cómo leer Access.

Es la separación entre cinco conceptos diferentes:

FUENTE
dato original

SOURCE ITEM
representación preservada de una unidad de la fuente

KNOWLEDGE ITEM
conocimiento normalizado que el sistema decide representar

EVIDENCE
explicación de por qué ese conocimiento existe

IMPORT RUN
registro de cómo y cuándo llegó la información

Esa separación será la base sobre la que se incorporarán progresivamente las técnicas de IA del laboratorio.

26. Estado al cierre

Repositorio local esperado:

/Users/javiercercosbudia/Desarrollo/aklab

Nombre del proyecto:

Applied Knowledge Lab

Nombre corto del repositorio:

aklab

Fase cerrada:

F0.2 - Foundation e ingesta inicial de SAT

Siguiente fase:

F1 - CRUD de conocimiento + entrada manual desde cero